Iterative vs Recursive Solutions

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Definition of Recursion
2. Factorial
3. Fibonacci Numbers
4. Summary

What is Recursion?

- A **recursive function** is one that calls itself directly or indirectly.
- Useful when a problem can be divided into smaller subproblems of the same kind.
- Every recursion must have:
  - a **base case** (stopping condition)
  - a **recursive case** (smaller version of the problem)

Example: Computing factorial or Fibonacci numbers.

Definition of Factorial

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1, \quad 0! = 1$$

Sample Input & Output:

$$\text{Input: } n = 5 \quad \Rightarrow \quad \text{Output: } 5! = 120$$

Recursive Formulation:

$$n! = \begin{cases} 1, & n = 0 \\ n \times (n-1)!, & n > 0 \end{cases}$$

C++ Code: Recursive Factorial

```cpp
#include <iostream>
using namespace std;

int factorial(int n){
  if(n == 0) return 1;        // base case
  return n * factorial(n-1); // recursive call
}

int main(){
  cout << factorial(5);  // prints 120
}
```

Factorial: Call Stack Trace (n = 4)

Step	Function call / Return value
1	factorial(4) = 4 × factorial(3)
2	factorial(3) = 3 × factorial(2)
3	factorial(2) = 2 × factorial(1)
4	factorial(1) = 1 × factorial(0)
5	factorial(0) = 1 (base case)
Return values:	$1 \to 1 \to 2 \to 6 \to 24$

Iterative Solution for Factorial

```cpp
#include <iostream>
using namespace std;

int factorial_iter(int n){
  int result = 1;
  for(int i=1; i<=n; i++){
    result = result * i;
  }
  return result;
}

int main(){
  cout << factorial_iter(5); // prints 120
}
```

Time and Space Complexity (Factorial)

Approach	Time Complexity	Space Complexity
Recursive	$O(n)$	$O(n)$ (call stack)
Iterative	$O(n)$	$O(1)$ (loop only)

Observation: Both take linear time, but recursion uses extra stack space.

Definition of Fibonacci Numbers

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \cdots\cdots$$

Sample Input & Output:

$$\text{Input: } n = 6 \quad \Rightarrow \quad \text{Output: } F(6) = 8$$

Recursive Formulation:

$$F(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F(n-1) + F(n-2), & n > 1 \end{cases}$$

C++ Code: Recursive Fibonacci (Vanilla)

```cpp
#include <iostream>
using namespace std;

int fib(int n){
  if(n == 0) return 0;
  if(n == 1) return 1;
  return fib(n-1) + fib(n-2);
}

int main(){
  cout << fib(6); // prints 8
}
```

C++ Code: Recursive Fibonacci (Memoized)

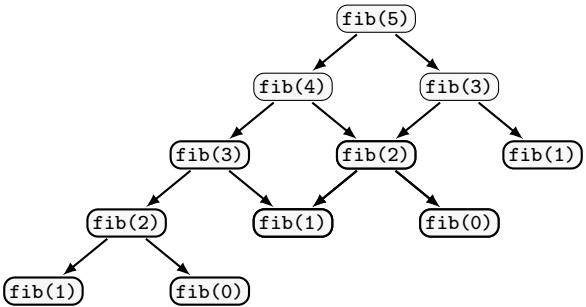
```cpp
#include <iostream>
using namespace std;

int memo[50];

int fib_memo(int n){
  if(n == 0) return 0;
  if(n == 1) return 1;
  if(memo[n] != -1) return memo[n];
  memo[n] = fib_memo(n-1) + fib_memo(n-2);
  return memo[n];
}

int main(){
  for(int i=0;i<50;i++) memo[i] = -1;
  cout << fib_memo(6); // prints 8
}
```

Fibonacci: Function Call Tree for $n = 5$ (TikZ)



Observation: Identical subcalls like `fib(3)` and `fib(2)` reappear many times, showing overlapping subproblems.

Iterative Solution for Fibonacci

```cpp
#include <iostream>
using namespace std;

int fib_iter(int n){
  if(n <= 1) return n;
  int a = 0, b = 1, c;
  for(int i=2; i<=n; i++){
    c = a + b;
    a = b;
    b = c;
  }
  return b;
}

int main(){
  cout << fib_iter(6); // prints 8
}
```

Time and Space Complexity (Fibonacci)

Approach	**Time Complexity**	**Space Complexity**
Recursive (vanilla)	$O(2^n)$	$O(n)$ (stack)
Recursive (memoized)	$O(n)$	$O(n)$ (array + stack)
Iterative	$O(n)$	$O(1)$ (loop only)

Observation: Memoization or iteration drastically reduces time cost.

Summary

- Recursion simplifies problem definition but adds stack overhead.
- Iteration often achieves the same result with constant space.
- For overlapping subproblems (like Fibonacci), use memoization or iteration.
- Always define a base case to prevent infinite recursion.